

Type Conversion, Floats, Complex, Rune, Strings

1. Explicit Type Conversions

In Go, you cannot accidentally mix different types of data, even if they look similar.

- **The Problem:** If you have an **int16** and an **int32**, the compiler sees them as two completely different types. If you try to say **x = y**, the program will fail to build.
- **The Solution:** You must explicitly convert the type using the syntax **Type(variable)**.
- **Example: x = int32(y).** Under the hood, Go takes the **16 bits** of y, safely adds **16** extra zeros (or ones, depending on the sign) to the front, and transforms it into a **32-bit number**.

2. Floating-Point and Complex Numbers

For real numbers (numbers with decimals), Go provides floating-point types.

- **float32:** Gives you about **6 decimal digits of precision**. Uses less memory.
- **float64:** Gives you about **15 decimal digits of precision**.
- **Best Practice:** Always default to **float64** unless you have a strict memory constraint. Floating-point math can sometimes suffer from microscopic rounding errors, so having higher precision is safer.
- **Complex Numbers:** Go natively supports advanced math! You can create complex numbers using the built-in function: **complex(realPart, imaginaryPart)**.

3. How Computers Read Text: ASCII vs. Unicode

To understand strings in Go, we must understand how computers translate 1s and 0s into letters.

- **ASCII (8-bit):** The original standard. It uses exactly **8 bits (1 byte)** per character. Because **8** bits can only represent **256 unique symbols**, it is only big enough to hold the English alphabet, numbers, and basic punctuation.
- **Unicode (32-bit):** To support global languages (**like Chinese, Arabic, or even emojis**), we needed a much bigger system. Unicode has enough room for billions of characters (**2³²**).

4. Variable-length encoding (UTF-8, UTF-16, UTF-32)

Variable-length encoding is a way to save memory by assigning different amounts of space (bytes) to different characters, rather than forcing every character to take up the exact same amount of space.

To understand why this is genius, look at the alternative: **Fixed-length encoding**. If a computer system used a fixed length of **4 bytes** per character just to ensure it had enough room for complex emojis, a simple word like **"cat"** would waste **12 bytes** of memory.

Variable-length encoding (like UTF-8, which Go uses natively) adapts dynamically to the character:

- Standard English letters (a, b, c) and numbers take just **1 byte**.
- More complex characters (like Greek or Arabic) take **2 bytes**.

- Heavy characters (like Chinese Hanzi or Emojis) take **3 to 4 bytes**.

The Engineering Benefit: Because Go uses this under the hood, your web apps and backend services can support every language and emoji on the planet without wasting massive amounts of RAM on standard English text. It makes your application globally compatible and highly memory-efficient at the exact same time.

4. Runes (Go's Concept of a Character)

Because **UTF-8** characters can be different sizes, Go uses a special concept called a **Rune**.

- A rune is simply Go's terminology for a single Unicode "**Code Point**" (one complete character).
- Even if a character takes **4 bytes of memory**, Go identifies that entire block as a single rune.

5. Strings in Go

In Go, a String (e.g., "hello world") is actually just a **sequence of bytes** encoded in **UTF-8**.

- **Strings are Read-Only (Immutable):** Once you create a string in Go, you cannot change the individual characters inside of it. If you want to modify a string, Go actually forces you to create a brand-new string behind the scenes.

Expert Takeaway:

Why does Go make strings completely read-only (immutable)? It all goes back to **Concurrency**! In a highly scalable backend, you might have thousands of **Goroutines** reading the exact same string (like a database connection URL or a JSON configuration) at the exact same time.

Because the string cannot be altered, you completely eliminate the risk of one Goroutine accidentally overwriting the text while another Goroutine is trying to read it. This makes Go web servers incredibly stable!